Uso abanzado de clases: Definición de funciones y uso de estas

En este paso vamos a describir **qué clases existen**, **qué métodos debe tener cada una** y **para qué sirve cada método**, de forma que entiendas claramente la **responsabilidad por capas** (app / model / service / repository / util).

---

0) Estructura objetivo del proyecto

```
src/main/java/com/docencia/
 ├── app/
 │    └── Main.java
 ├── model/
 │    ├── Persona.java
 │    └── Usuario.java
 ├── util/
 │    └── Validaciones.java
 ├── repository/
 │    └── IUserRepository.java
 ├── repository/impl/
 │    └── UserRepositoryImpl.java
 ├── service/
 │    ├── IAuthService.java
 │    └── IUserService.java
 └── service/impl/
      ├── AuthServiceImpl.java
      └── UserServiceImpl.java
```

> Nota: Recueda que el `Main` **NO** debe acceder a colecciones ni a datos directamente; solo llama a `service`.

---

1) Responsabilidad por capas

app

- Menú, lectura por teclado, mostrar mensajes.

- Llama a **interfaces** de servicio.

model

- Entidades (datos) y reglas “locales” (básicas). No piensa.
- NO contiene lógica de login/registro (eso es service).

util

- Validaciones reutilizables (email, password, normalización).

repository

- Acceso a datos (en esta práctica: memoria).
- CRUD y búsquedas. Sin reglas de negocio. Los métodos que ya conocemos

service

- Casos de uso (registro, login, bloqueo, búsqueda).
- Aplica reglas de negocio, usa repository y util.

---

2) Métodos por clase (qué vas a definir y por qué)

A continuación tienes una lista “contractual” de métodos. Puedes ajustarla mínimamente, pero **si cambias nombres**, cambia también tests y menú.

2.1 `model/Persona.java` (abstracta)

Responsabilidad: base común para modelos (id/nombre). No se instancia.

Campos recomendados

- `protected final int id;`
- `protected String nombre;`

Métodos a definir

- `protected Persona(int id, String nombre)`
  Función: constructor base. Valida `id > 0` y `nombre` (mínimo 5 caracteres según enunciado).
- `protected Persona(int id)`
- `protected Persona(int id, String nombre)`. Todos los constructores deben de tener las validaciones mínimas (propidades obligatorias).
- `public int getId()`
  Función: devuelve id.
- `public String getNombre()`
  Función: devuelve nombre.

- `public void setNombre(String nombre)`
  Función: actualiza nombre con validación.
- `@Override public String toString()`
  Función: representación para listados.

> Regla: las validaciones de `id`/`nombre` pueden estar aquí para reutilizarse en Usuario.

2.2 `model/Usuario.java` (extiende Persona)

Responsabilidad: entidad "Usuario" + estado de seguridad (intentos y bloqueo).

Campos mínimos (como el enunciado)

- `private final String email;`
- `private String password;`
- `private int intentosFallidos;`
- `private boolean bloqueado;`
- `private final java.time.LocalDate fechaRegistro;`

Métodos a definir

- `public Usuario(int id, String nombre, String email, String password)`
  Función: crea usuario válido. Normaliza email (`trim + lower`) y valida:
  - nombre (>= 5 caracteres)
  - email no vacío y formato válido (delegando en `Validaciones`)
  - password mínima 6 (delegando en `Validaciones`)
  - inicializa `intentosFallidos=0`, `bloqueado=false`, `fechaRegistro=LocalDate.now()`
- `public Usuario(String email)`. Las validaciones mínimas deben de ser las mismas que en caso anterior.
- `public String getEmail()`
  Función: devuelve email normalizado (clave lógica).
- `public String getPassword()`
  Función: devuelve password.
- `public void setPassword(String nuevaPassword)`
  Función: cambio de contraseña validando reglas.
- `public int getIntentosFallidos()`
  Función: consulta intentos.
- `public boolean isBloqueado()`
  Función: consulta bloqueo.
- `public void incrementarIntentosFallidos()`
  Función: +1 intento (NO decide bloqueo; eso es regla de negocio del servicio).
- `public void resetearIntentosFallidos()`
  Función: pone intentos a 0.
- `public void bloquear()`
  Función: marca usuario como bloqueado.
- `@Override public boolean equals(Object o)` y `hashCode()`
  Función: que `Set<Usuario>` o `Map` funcione por email (recomendado: igualdad por email).

- `@Override public String toString()`
  Función: mostrar datos `sin exponer password` (no mostrar password).

2.3 `util/Validaciones.java` (clase static)

Responsabilidad: validaciones reutilizables. No guarda estado.

Métodos a definir

- `public static String normalizarEmail(String email)`
  Función: `trim()` + `toLowerCase()`. Devuelve null si entra null (o lanza excepción; decide una estrategia).
- `public static boolean emailValido(String email)`
  Función: true si cumple regex básica. Ejemplo recomendado:
  - `^[^@\s]+@[^@\s]+\.[^@\s]+$`
- `public static boolean passwordValida(String password)`
  Función: longitud mínima 6.
- `public static void validarNombre(String nombre)`
  Función: lanza `IllegalArgumentException` si nombre < 5.
- `public static void validarEmail(String email)`
  Función: lanza `IllegalArgumentException` si email inválido.
- `public static void validarPassword(String password)`
  Función: lanza `IllegalArgumentException` si password inválida.

> Consejo docente: separar `boolean` y `void validar...` ayuda a tests y a lanzar errores consistentes.

2.4 `repository/IUserRepository.java` (interfaz)

Responsabilidad: contrato CRUD/búsqueda. Sin reglas de negocio.

Métodos a definir

- `void save(Usuario usuario)`
  Función: almacena usuario.
- `java.util.Optional<Usuario> findByEmail(String email)`
  Función: busca por email (entrada puede venir con mayúsculas/espacios; internamente normalizar).
- `boolean existsByEmail(String email)`
  Función: true si existe email.
- `java.util.Set<Usuario> findAll()` (o `List<Usuario>`)
  Función: devuelve todos.
- `boolean deleteByEmail(String email)`
  Función: elimina por email, devuelve true si eliminó.

2.5 `repository/impl/UserRepositoryImpl.java`

Responsabilidad: implementación en memoria del repositorio.

Estructura recomendada

Opción A (muy clara y eficiente): `Map<String, Usuario>` donde clave es email normalizado.

- `private final java.util.Map<String, Usuario> usuarios = new java.util.HashMap<>();`

Opción B (si quieres practicar `Set` como en el enunciado): `Set<Usuario>`

- `private final java.util.Set<Usuario> usuarios = new java.util.HashSet<>();`
- Requiere `equals/hashCode` por email en `Usuario`.

Reglas dentro del repo

- No mezclar reglas de login/bloqueo.
- Sí mantener integridad del almacenamiento (p.ej. duplicados).

Métodos (implementan la interfaz)

- `save(...)`
  Función: guarda usuario; si ya existe email → `IllegalArgumentException`.
- `findByEmail(...)`
  Función: normaliza email y busca.
- `existsByEmail(...)`
  Función: normaliza email y comprueba existencia.
- `findAll()`
  Función: devuelve copia/colección inmodificable si quieres evitar cambios externos.
- `deleteByEmail(...)`
  Función: elimina por email normalizado.

2.6 `service/IUserService.java` (interfaz)

Responsabilidad: casos de uso “CRUD” de usuarios (gestión).

Métodos a definir

- `Usuario crearUsuario(int id, String nombre, String email, String password)`
  Función: crea y guarda (validando) mediante repo.
- `java.util.Set<Usuario> listarUsuarios()`
  Función: lista usuarios (modo debug).
- `java.util.Optional<Usuario> buscarPorEmail(String email)`
  Función: buscar por email.
- `boolean eliminarPorEmail(String email)`
  Función: borrar por email.
- `Usuario cambiarNombre(String email, String nuevoNombre)` *(opcional)*
  Función: ejemplo de update.

- `Usuario cambiarPassword(String email, String nuevaPassword)` *(opcional)*
  Función: ejemplo de update.

2.7 `service/impl/UserServiceImpl.java`

Responsabilidad: implementación de `IUserService` usando repository y util.

Dependencias (inyección por constructor)

- `private final IUserRepository repo;`

Comportamiento

- Valida entradas con `Validaciones`.
- Llama al repo para persistir/consultar.
- No gestiona login (eso es `AuthService`).

2.8 `service/IAuthService.java` (interfaz)

Responsabilidad: casos de uso de autenticación (registro + login).

Métodos a definir

- `Usuario register(int id, String nombre, String email, String password)`
  Función: registra usuario. (Puede delegar en `IUserService` o usar repo directamente.)
- `boolean login(String email, String password)`
  Función: devuelve true si acceso permitido.
- `boolean isBloqueado(String email)` *(opcional)*
  Función: consulta bloqueo (útil para menú).
- `void desbloquear(String email)` *(ampliación)*
  Función: desbloqueo manual (nivel 2).

2.9 `service/impl/AuthServiceImpl.java`

Responsabilidad: aplicar reglas de negocio de login/bloqueo.

Dependencias

Opción A (simple): usa repository directamente

- `private final IUserRepository repo;`

Opción B (más “capas puras”): depende de `IUserService`

- `private final IUserService userService;`

Reglas de negocio (obligatorias)

- Email inexistente → login false.

- Usuario bloqueado → login false.
- Password incorrecta → incrementa intentos; al 3º fallo → bloquear.
- Password correcta → reset intentos y login true.

Métodos

- `register(...)`
  Función: valida y crea usuario, evitando duplicados.
- `login(email, password)`
  Función: aplica las reglas anteriores, actualizando el usuario.
- *(opcional)* `isBloqueado(email)`
  Función: consulta estado actual.

2.10 `app/Main.java`

Responsabilidad: interfaz por consola.

Métodos recomendados

- `public static void main(String[] args)`
  Función: arranca la app y muestra menú en bucle.
- `private static void mostrarMenu()`
  Función: imprime opciones.
- `private static int leerOpcion()`
  Función: lee int con control de errores.
- `private static void registrar(IAuthService auth)`
  Función: pide datos y llama a `auth.register(...)`.
- `private static void login(IAuthService auth)`
  Función: pide email/password y llama a `auth.login(...)`.
- `private static void listar(IUserService users)`
  Función: imprime listado.
- `private static void buscar(IUserService users)`
  Función: busca por email.
- `private static void eliminar(IUserService users)`
  Función: borra por email.

> Importante: **Main no valida con regex** (eso va a `Validaciones`) y **no maneja colecciones** (eso va a service/repo).

---

3) Construcción paso a paso (orden recomendado)

Paso 1 — Crea el proyecto y los paquetes

1. Crea proyecto Maven/Java.
2. Crea paquetes según estructura.
3. Compila en blanco.

Checklist:

- [ ] Compila sin clases implementadas (solo estructura).

Paso 2 — Implementa `Validaciones`

1. Crea `Validaciones.normalizarEmail()`
2. Crea `emailValido()` y `passwordValida()`
3. Crea `validarNombre/Email/Password()` lanzando `IllegalArgumentException`.

Checklist:

- [ ] Puedes validar emails y passwords con tests.

Paso 3 — Implementa `Persona` y `Usuario`

1. `Persona` abstracta con validación de id/nombre.
2. `Usuario` extiende Persona y añade campos.
3. Normaliza email en constructor usando `Validaciones.normalizarEmail()`.
4. Implementa `equals/hashCode` por email.
5. No muestres password en `toString()`.

Checklist:

- [ ] Se crean usuarios válidos.
- [ ] Se lanzan `IllegalArgumentException` cuando corresponde.

Paso 4 — Implementa `IUserRepository` + `UserRepositoryImpl`

1. Define interfaz con CRUD.
2. Implementa con `Map` o `Set`.
3. Asegura “email único”.

Checklist:

- [ ] `save` no permite duplicados.
- [ ] `findByEmail` funciona con email en mayúsculas/espacios.

Paso 5 — Implementa `IUserService` + `UserServiceImpl`

1. Constructor con `IUserRepository`.
2. `crearUsuario` → crea `Usuario` y guarda en repo.
3. `listar/buscar/eliminar` delegan en repo.

Checklist:

- [ ] CRUD básico funcionando.

Paso 6 — Implementa `IAuthService` + `AuthServiceImpl`

1. `register` llama a `crearUsuario` o repo.
2. `login` aplica reglas de intentos y bloqueo.
3. Actualiza usuario: incrementa/reset/bloquea.

Checklist:

- [ ] 3 fallos bloquean.
- [ ] usuario bloqueado no puede loguearse.

Paso 7 — Implementa `Main` (menú)

1. Crea instancias:
   - `IUserRepository repo = new UserRepositoryImpl();`
   - `IUserService userService = new UserServiceImpl(repo);`
   - `IAuthService authService = new AuthServiceImpl(repo /* o userService */);`
2. Implementa menú mínimo:
   - Registrar
   - Login
   - Listar
   - Buscar
   - Eliminar
   - Salir

Checklist:

- [ ] No hay lógica de negocio dentro de Main.
- [ ] Manejo de errores con try/catch.

---

4) Ejemplo de flujo (para explicar en clase)

- `Main` pide email/password y llama a `authService.login(email, pass)`
- `AuthServiceImpl`:
  - normaliza email
  - busca usuario en repo
  - comprueba bloqueo
  - compara password
  - incrementa intentos o resetea
  - bloquea si corresponde
- Repo solo guarda/busca.

---

5) Ejecución

Compilar y testear

```
mvn clean test
```

Ejecutar app (si usas plugin o IDE)

- Ejecuta `Main` desde el IDE
- o usa `mvn exec:java` si lo configuras